NEURAL NETWORK LAB PROJECT REPORT

ON

MNIST Digit Classifier

Handwritten Digit Recognition using CNN with PyQt5 GUI Interface

In partial fulfillment of

B.Tech. III yr (Computer Science & Engineering)

Submitted To:

Dr. Naresh Purohit

HOD CSE Dept.

Submitted By:

Vishnu Kant

Roll No: 23EJICS182

Jodhpur Institute of Engineering and Technology

Department of Computer Science & Engineering

Session 2025–26

Acknowledgment

I would like to acknowledge the contributions of the following people without whose help and guidance this project would not have been completed successfully.

I respectfully thank **Naresh Purohit Sir**, for providing me an opportunity to undertake this project and giving me all support and guidance, which enabled me to complete this work to a great extent.

I am also thankful to **Prof. Dr. Anju Jangid**, Head of the Computer Science and Engineering Department, Jodhpur Institute of Engineering and Technology, for constant encouragement, valuable suggestions, moral support, and blessings.

Although it is not possible to name individually, I shall ever remain indebted to the faculty members of Jodhpur Institute of Engineering and Technology for their persistent support and cooperation extended during this work.

This acknowledgment will remain incomplete if I fail to express my deep sense of obligation to my parents and God for their consistent blessings and encouragement.

(VISHNU KANT)

Abstract

Handwritten digit recognition is one of the foundational problems in the field of computer vision and machine learning. Accurate recognition of handwritten digits has practical applications in postal automation, bank cheque processing, form digitization, and educational technology. Despite its apparent simplicity, building a system that achieves human-level accuracy on digit recognition requires carefully designed neural network architectures and training strategies.

MNIST Digit Classifier is a deep learning project that implements a Convolutional Neural Network (CNN) in PyTorch to classify handwritten digits from the MNIST dataset. The model is trained on 60,000 grayscale images and evaluated on 10,000 test images spanning 10 digit classes (0–9), each of size 28×28 pixels. The CNN architecture consists of two convolutional layers with ReLU activations, max pooling, dropout regularization, and two fully connected layers, achieving a test accuracy of approximately **99%** after 14 epochs of training.

The project uses **PyTorch** as the primary deep learning framework with the **torchvision** library for dataset loading and image transforms. Training is performed using the **Adadelta** optimizer with a **StepLR** learning rate scheduler. The trained model weights are saved to `mnist_cnn.pt` for future inference. The codebase is structured as a `main.py` training script with modular `train()` and `test()` functions and a full command-line argument interface.

In addition to the training pipeline, the project includes a fully functional **PyQt5 graphical user interface** implemented in `modern_gui.py`. The GUI allows users to load any handwritten digit image from disk or via drag-and-drop, automatically resizes it to 28×28 pixels, applies the same preprocessing transforms used during training, and runs inference on the saved model using a background `QThread` worker. The prediction result, confidence percentage, and a full bar chart of all 10 class probabilities are displayed in real time within the application window.

Keywords: Convolutional Neural Network, MNIST, PyTorch, Handwritten Digit Recognition, Image Classification, Adadelta, Dropout, StepLR, Deep Learning, torchvision,

PyQt5, GUI, QThread, Drag and Drop.

Contents

Acknowledgment **1**

Abstract **2**

List of Figures **6**

List of Tables **7**

1 Introduction **8**

1.1 Background and Project Overview 8

1.2 Objective 9

1.3 Problem Statement 9

1.4 Real-World Applications of Digit Recognition 10

1.5 Scope 10

2 Technology Used in Project **12**

2.1 Python and PyTorch 12

2.2 Core Libraries and Modules 12

2.3 GUI Framework — PyQt5 and Pillow 13

2.4 Development Tools and Environment 14

2.5 Project Folder Structure 14

3 Technical Details of Project **15**

3.1 System Architecture 15

3.2 CNN Architecture — The Net Class 15

3.2.1 Layer-by-Layer Dimension Analysis 17

3.3 Training Implementation 17

3.4 System Flowchart 18

3.5 Optimizer and Learning Rate Scheduler 18

3.6 Data Preprocessing Pipeline 19

3.7 Module Descriptions 19

3.8 GUI Architecture and Key Implementation 20

4 Output and Test Cases **22**

4.1 Test Case 1: Default Training Run (14 Epochs, CPU) 22

4.2 Test Case 2: Per-Epoch Training Loss and Accuracy 22

4.3 Test Case 3: GPU Acceleration vs CPU 23

4.4 Test Case 4: Effect of Batch Size on Training 23

4.5 Test Case 5: Dropout Disabled (Overfitting Demonstration) 24

4.6 Test Case 6: Model Saving and Loading 24

4.7 Test Case 7: GUI — Load Image and Predict 25

4.8 Test Case 8: GUI — Drag and Drop + Model Not Found Error 26

4.9 Command-Line Interface Examples 27

4.10 Syllabus Coverage 28

4.11 Project Screenshots 28

5 Conclusion **31**

5.1 Key Achievements 31

5.2 Technical Skills Acquired 32

5.3 Lessons Learned 32

6 Future Work **34**

7 References **36**

List of Figures

3.1 MNIST Digit Classifier Complete System Flowchart 18

4.1 MNIST Digit Classifier GUI — Main Window with No Image Loaded . . 28

4.2 GUI — Image Loaded and Digit Predicted with Confidence Score 29

4.3 GUI — Class Probability Bars Showing Distribution across All 10 Digits 29

4.4 Terminal Output — CNN Training Progress with Per-Epoch Loss and Accuracy 30

List of Tables

1.1 Real-World Applications of Handwritten Digit Recognition 10

2.1 Development Tools and Versions 14

3.1 CNN Layer-by-Layer Dimension Analysis 17

4.1 Test Case 1 — Default Training Configuration 22
4.2 Test Case 2 — Per-Epoch Loss and Accuracy Progression 23
4.3 Test Case 3 — Hardware Acceleration Comparison 23
4.4 Test Case 4 — Effect of Batch Size on Model Performance 24
4.5 Test Case 5 — Dropout Effect: Overfitting Demonstration 24
4.6 Test Case 6 — Model Saving and Loading 25
4.7 Test Case 7 — GUI Image Load and Prediction 25
4.8 Test Case 8a — Drag and Drop Image Loading 26
4.9 Test Case 8b — Model Not Found Error Handling 26
4.10 CNN vs Fully Connected Network on MNIST 26
4.11 MNIST Digit Classifier Syllabus Coverage Map 28

1

Introduction

1.1 Background and Project Overview

Handwritten digit recognition has been a benchmark problem in pattern recognition and machine learning since the 1980s. The ability of a computer to correctly identify handwritten numerals written by different people with varying styles, sizes, and orientations is a fundamental challenge that requires the system to learn abstract, hierarchical feature representations from raw pixel data.

The **MNIST** (Modified National Institute of Standards and Technology) dataset, introduced by Yann LeCun et al. in 1998, is the most widely used benchmark dataset for this problem. It contains 70,000 grayscale images of handwritten digits collected from American Census Bureau employees and American high school students, making it highly representative of real-world variation in handwriting styles.

MNIST Digit Classifier is a deep learning project that implements a Convolutional Neural Network (CNN) using the PyTorch framework to classify these handwritten digits with high accuracy. The system trains on 60,000 images and achieves approximately 99% accuracy on the held-out test set of 10,000 images. Beyond the core training pipeline, the project includes a **PyQt5 graphical user interface** (`modern_gui.py`) that allows any user to load a custom handwritten digit image and instantly receive the predicted digit along with a full probability distribution across all 10 classes — without any command-line interaction.

Key Concept — What is a Convolutional Neural Network (CNN)?

A Convolutional Neural Network is a type of deep neural network specifically designed for processing grid-structured data such as images. Unlike fully connected networks, CNNs use **convolutional layers** that slide small learnable filters (kernels) across the input image to detect local features such as edges, curves, and textures. These features are progressively combined in deeper layers to recognize complex patterns such as digit shapes. The key advantage is **parameter sharing**

— the same filter is applied at every position, drastically reducing the number of trainable parameters compared to a fully connected layer.

1.2 Objective

- Implement a CNN-based image classifier for the MNIST handwritten digit dataset using PyTorch.
- Achieve test accuracy of 99% or above after 14 epochs of training with the Adadelta optimizer.
- Demonstrate a complete deep learning pipeline: data loading, preprocessing, model definition, training, evaluation, and model saving.
- Apply dropout regularization to prevent overfitting and StepLR scheduling to improve convergence.
- Build a **PyQt5 desktop GUI** (`MnistGUI`) that loads any image file, preprocesses it, runs inference using the saved model via a background `QThread`, and displays the predicted digit, confidence score, and class probability bars.
- Support drag-and-drop image loading and background threading to keep the GUI responsive during prediction.
- Cover practical implementation of Machine Learning syllabus topics including CNNs, backpropagation, regularization, model evaluation, and GUI development.

1.3 Problem Statement

Handwritten digit recognition is a multi-class image classification problem. Given a 28×28 grayscale image of a handwritten digit, the system must correctly identify which of the 10 digit classes (0 through 9) it belongs to. The challenges include:

1. **Intra-class variation:** The same digit (e.g., the digit "7") can be written in many different styles, sizes, and slants by different people.
2. **Feature extraction:** Raw pixel values are poor features — the network must learn hierarchical spatial features automatically from training examples.

3. **Generalisation:** The model must perform well on unseen handwriting styles in the test set, not just memorise the training data (overfitting avoidance).

1.4 Real-World Applications of Digit Recognition

Application	How Digit Recognition Is Used
Postal Automation	Automatic reading of PIN codes on envelopes to route mail without human intervention.
Bank Cheque Processing	Reading handwritten amounts on cheques for automated clearance and fraud detection.
Form Digitization	Converting handwritten paper forms (tax returns, applications) to structured digital data.
Exam Answer Evaluation	Automated grading of fill-in-the-bubble and handwritten numerical answers in OMR sheets.
Medical Records	Digitizing handwritten patient data, dosage values, and lab results in hospital records.
Captcha Verification	Some CAPTCHA systems use handwritten-style digits to distinguish humans from bots.

Table 1.1: Real-World Applications of Handwritten Digit Recognition

1.5 Scope

Functional Scope:

- CNN training on 60,000 MNIST training images with automatic dataset download via `torchvision`.
- Per-epoch evaluation on 10,000 test images with average loss and accuracy reporting.
- Command-line interface for all hyperparameters: batch size, epochs, learning rate, gamma, seed, log interval.
- Optional GPU/CPU acceleration via PyTorch device detection (CUDA, MPS, CPU).

- Model persistence — saving trained weights to `mnist_cnn.pt` with `-save-model` flag.
- **PyQt5 desktop GUI** with image file browser, drag-and-drop support, 900 × 700 px window, live prediction display, confidence percentage, and animated probability bars for all 10 digit classes.

Technical Scope:

- Deep Learning: PyTorch CNN with 2 convolutional layers, 2 dropout layers, 2 fully connected layers.
- Optimisation: Adadelta optimizer (lr=1.0) with StepLR scheduler (gamma=0.7, step size=1).
- Data Pipeline: torchvision MNIST dataset with ToTensor transform and normalisation (mean=0.1307, std=0.3081).
- Loss Function: Negative Log-Likelihood Loss (`F.nll_loss`) with log-softmax output.
- GUI Framework: PyQt5 (`QMainWindow`, `QThread`, `pyqtSignal`) with Pillow for image preprocessing.

Exclusions: The project does **not** include a web-based interface or REST API deployment, does not support real-time webcam input, and does not implement a canvas for drawing digits directly within the application.

2

Technology Used in Project

2.1 Python and PyTorch

Python 3 is the primary programming language, chosen for its rich ecosystem of scientific computing and deep learning libraries. **PyTorch** is an open-source deep learning framework developed by Meta AI Research that provides dynamic computation graphs, automatic differentiation via `autograd`, and seamless GPU acceleration. PyTorch was selected because it offers an intuitive, Pythonic API for building and training neural networks, and is the most widely adopted framework in academic deep learning research.

2.2 Core Libraries and Modules

- **torch.nn:** Provides all neural network building blocks including `nn.Module` (base class), `nn.Conv2d` (convolutional layers), `nn.Linear` (fully connected layers), and `nn.Dropout` (regularisation).
- **torch.nn.functional:** Stateless functions used in the forward pass: `F.relu` (activation), `F.max_pool2d` (pooling), `F.log_softmax` (output), and `F.nll_loss` (loss computation).
- **torch.optim:** Optimisation algorithms module. Used for `optim.Adadelta` (the primary optimizer) which adapts learning rates per parameter using accumulated gradient information.
- **torch.optim.lr_scheduler.StepLR:** Learning rate scheduler that multiplies the learning rate by `gamma=0.7` at the end of every epoch, enabling the model to take large steps early in training and fine-tune with smaller steps later.
- **torchvision.datasets.MNIST:** Provides automatic download, caching, and loading of the MNIST dataset in PyTorch `Dataset` format.

- **torchvision.transforms:** Preprocessing pipeline: `transforms.ToTensor()` converts PIL images to float tensors in range [0,1]; `transforms.Normalize((0.1307,), (0.3081,))` standardises pixel values using the MNIST dataset mean and standard deviation.

- **torch.utils.data.DataLoader:** Wraps the dataset to provide automatic batching, shuffling, and multi-process data loading during training.

2.3 GUI Framework — PyQt5 and Pillow

- **PyQt5:** Python bindings for the Qt5 framework used to build the complete desktop GUI. Key classes used include `QMainWindow` (main application window, 900 × 700 px), `QWidget` and layout managers (`QVBoxLayout`, `QHBoxLayout`) for responsive two-column layout, `QLabel` for image display and text output, `QPushButton` for Load/Clear/Exit actions, `QProgressBar` for the 10 class probability bars, `QFileDialog` for the image file browser, and `QFrame` for styled card panels.

- **QThread and pyqtSignal:** The `PredictionWorker` class inherits `QThread` and runs the CNN inference in a separate background thread, preventing the GUI from freezing during prediction. It emits `prediction_done(int, float, list)` on success and `prediction_error(str)` on failure via `pyqtSignal`, which are connected to the main thread's update slots.

- **Pillow (PIL):** Used in both `predict_image()` and `process_image()` to open any image format (`.png`, `.jpg`, `.bmp`, `.gif`, `.tiff`), convert it to grayscale (`.convert("L")`), and resize to 28 × 28 pixels for model input and 280 × 280 pixels for the GUI preview display.

- **argparse:** Python standard library module for the command-line training interface in `main.py`.

- **torch.cuda / torch.backends.mps:** Automatic hardware detection. The GUI loads the model on the same device (CUDA/MPS/CPU) as is available, requiring no user configuration.

2.4 Development Tools and Environment

Tool	Version	Purpose
Python	3.8+	Primary programming language
PyTorch	2.0+	Deep learning framework and autograd
torchvision	0.15+	MNIST dataset loading and transforms
PyQt5	5.15+	Desktop GUI framework for `modern_gui.py`
Pillow (PIL)	9.0+	Image loading, grayscale conversion, resizing
pip	23+	Python package management
VS Code	Latest	Primary IDE with Python extension
Git	v2.x	Version control and code backup
CUDA (optional)	11.8+	GPU acceleration for faster training

Table 2.1: Development Tools and Versions

2.5 Project Folder Structure

```
mnist/
 main.py              <- CNN model (Net), train(), test(), main() + argpar
 modern_gui.py        <- PyQt5 GUI: MnistGUI, PredictionWorker, predict_im
 requirements.txt     <- torch, torchvision, PyQt5, Pillow dependencies
 mnist_cnn.pt         <- Saved model weights (generated by --save-model)
 data/                <- Auto-created; stores downloaded MNIST files
     MNIST/
         raw/         <- Raw binary MNIST image and label files
```

3

Technical Details of Project

3.1 System Architecture

The MNIST Digit Classifier follows a three-tier architecture combining a deep learning training pipeline with a desktop GUI inference layer:

- **Data Layer:** The MNIST dataset (60,000 training + 10,000 test images) is downloaded automatically via `torchvision.datasets.MNIST` and served in batches of 64 (training) and 1000 (testing) through `DataLoader` with shuffling enabled for training.
- **Model Layer:** A CNN defined as the `Net` class in `main.py` inheriting from `nn.Module`. The network processes $1 \times 28 \times 28$ input tensors through two convolutional blocks and two fully connected layers to produce a 10-class log-probability output. The trained weights are saved to `mnist_cnn.pt`.
- **GUI Presentation Layer:** The `MnistGUI` class in `modern_gui.py` provides a 900×700 px PyQt5 desktop window. It loads `mnist_cnn.pt` at startup, accepts image input via file dialog or drag-and-drop, preprocesses the image identically to training, runs inference on a background `QThread` worker, and updates the prediction display, confidence label, and 10 probability bars in real time.

3.2 CNN Architecture — The Net Class

The model architecture is defined as follows. Each layer's input and output dimensions are noted for clarity:

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1    = nn.Conv2d(1, 32, 3, 1)   # in: 1x28x28  out: 32x2
```

```
        self.conv2    = nn.Conv2d(32, 64, 3, 1)  # in: 32x26x26 out: 64x2
        self.dropout1 = nn.Dropout(0.25)
        self.dropout2 = nn.Dropout(0.5)
        self.fc1      = nn.Linear(9216, 128)     # 64x12x12 = 9216 after
        self.fc2      = nn.Linear(128, 10)       # 10 output classes (dig

    def forward(self, x):
        x = self.conv1(x)              # Conv layer 1
        x = F.relu(x)                  # ReLU activation
        x = self.conv2(x)              # Conv layer 2
        x = F.relu(x)                  # ReLU activation
        x = F.max_pool2d(x, 2)        # Max pooling (2x2)
        x = self.dropout1(x)           # Dropout 25%
        x = torch.flatten(x, 1)       # Flatten to 1D: 9216
        x = self.fc1(x)                # Fully connected 1
        x = F.relu(x)                  # ReLU activation
        x = self.dropout2(x)           # Dropout 50%
        x = self.fc2(x)                # Fully connected 2 (output)
        output = F.log_softmax(x, dim=1)  # Log-softmax for NLL loss
        return output
```

3.2.1 Layer-by-Layer Dimension Analysis

Layer	**Input Shape**	**Output Shape**	**Operation**
Input	$1 \times 28 \times 28$	$1 \times 28 \times 28$	Raw grayscale image
Conv1 + ReLU	$1 \times 28 \times 28$	$32 \times 26 \times 26$	32 filters, 3×3 kernel, stride 1
Conv2 + ReLU	$32 \times 26 \times 26$	$64 \times 24 \times 24$	64 filters, 3×3 kernel, stride 1
MaxPool2d	$64 \times 24 \times 24$	$64 \times 12 \times 12$	2×2 pool, stride 2
Dropout1	$64 \times 12 \times 12$	$64 \times 12 \times 12$	25% neurons zeroed
Flatten	$64 \times 12 \times 12$	9216	$64 \times 12 \times 12 = 9216$
FC1 + ReLU	9216	128	Fully connected
Dropout2	128	128	50% neurons zeroed
FC2	128	10	Output logits
Log-Softmax	10	10	Log-probabilities

Table 3.1: CNN Layer-by-Layer Dimension Analysis

3.3 Training Implementation

The training function iterates over batches, performs forward and backward passes, and updates weights:

```
def train(args, model, device, train_loader, optimizer, epoch):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()           # Reset gradients
        output = model(data)            # Forward pass
        loss = F.nll_loss(output, target)  # Compute NLL loss
        loss.backward()                 # Backpropagation
        optimizer.step()                # Update weights
        if batch_idx % args.log_interval == 0:
            print(f'Train Epoch: {epoch} Loss: {loss.item():.6f}')
```

3.4 System Flowchart

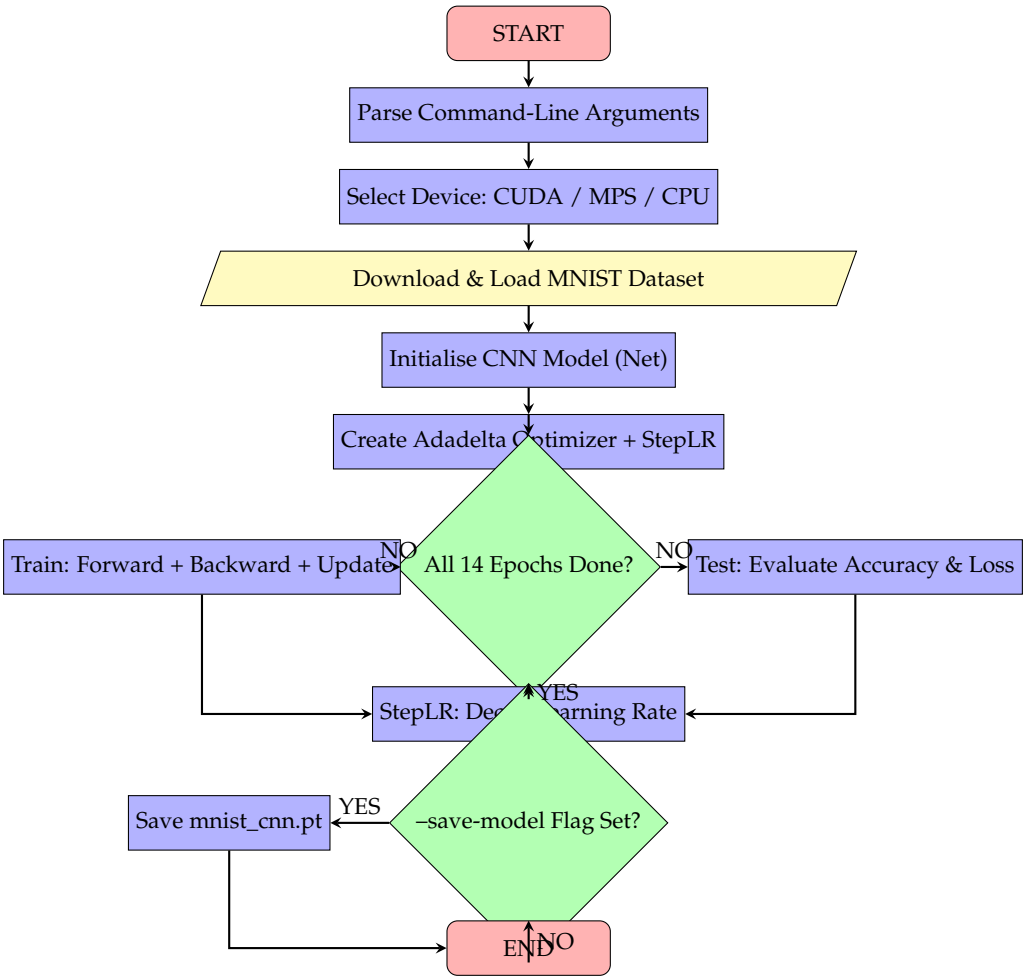

Figure 3.1: MNIST Digit Classifier Complete System Flowchart

3.5 Optimizer and Learning Rate Scheduler

```
# Adadelta optimizer with initial learning rate = 1.0
optimizer = optim.Adadelta(model.parameters(), lr=args.lr)

# StepLR: multiply lr by gamma=0.7 after every epoch
scheduler = StepLR(optimizer, step_size=1, gamma=args.gamma)

for epoch in range(1, args.epochs + 1):
    train(args, model, device, train_loader, optimizer, epoch)
    test(model, device, test_loader)
```

```
    scheduler.step()            # Decay learning rate

if args.save_model:
    torch.save(model.state_dict(), "mnist_cnn.pt")
```

Adadelta is an adaptive learning rate optimizer that requires no manual tuning of the learning rate decay — it computes per-parameter learning rates based on a window of past gradients. **StepLR** further decays the global learning rate by $\gamma = 0.7$ each epoch, allowing large gradient steps early in training and progressively finer updates near convergence.

3.6 Data Preprocessing Pipeline

```
transform = transforms.Compose([
    transforms.ToTensor(),              # PIL Image -> Float Tensor [0,1]
    transforms.Normalize((0.1307,), (0.3081,))  # Standardise pixels
])

train_loader = DataLoader(
    datasets.MNIST('../data', train=True,  download=True, transform=trans
    batch_size=64, shuffle=True)

test_loader = DataLoader(
    datasets.MNIST('../data', train=False, download=True, transform=trans
    batch_size=1000, shuffle=False)
```

The normalisation constants `mean=0.1307` and `std=0.3081` are the pre-computed global statistics of the MNIST training set, ensuring zero-mean unit-variance inputs which stabilise gradient flow during training.

3.7 Module Descriptions

- **Net (class) — main.py:** CNN model class inheriting `nn.Module`. Defines all layers in `__init__` and the forward computation graph in `forward()`. Con-

tains conv1, conv2, dropout1, dropout2, fc1, fc2. Imported by `modern_gui.py` for inference.

- **train() — main.py:** Runs one complete epoch over the training set. Sets `model.train()` mode, iterates batches, computes NLL loss, calls `loss.backward()`, and updates weights via `optimizer.step()`.
- **test() — main.py:** Evaluates the model on the full test set. Sets `model.eval()` mode, wraps inference in `torch.no_grad()`, accumulates correct predictions, and prints average loss and accuracy.
- **main() — main.py:** Entry point for training. Parses CLI arguments, detects device, creates data loaders, instantiates `Net`, runs the training loop, and saves model weights.
- **predict_image() — modern_gui.py:** Standalone inference function. Opens the image with Pillow, converts to grayscale, resizes to 28×28, applies the training transforms, runs a forward pass with `torch.no_grad()`, and returns the predicted class, confidence, and full 10-class probability list.
- **PredictionWorker (class) — modern_gui.py:** Inherits `QThread`. Runs `predict_image()` in a background thread and emits `prediction_done` or `prediction_error` signals to the main GUI thread, keeping the interface responsive.
- **MnistGUI (class) — modern_gui.py:** Main `QMainWindow` application. Two-column layout: left panel shows 300×300 image preview; right panel shows prediction digit (48pt bold), confidence label, and 10 `QProgressBar` probability bars (one per class). Supports drag-and-drop via `dragEnterEvent` and `dropEvent`.

3.8 GUI Architecture and Key Implementation

The GUI uses a **two-column layout**: the left column displays the loaded image preview in a 300×300 `QLabel` inside a dashed `QFrame`; the right column contains a prediction frame showing the predicted digit in 48pt bold blue font with confidence below it, and a probability frame with 10 rows of `QProgressBar` widgets (one per digit class 0–9).

The `predict_image()` function applies exactly the same preprocessing pipeline used during training:

```
def predict_image(image_path: str) -> tuple[int, float, list]:
    img = Image.open(image_path).convert("L")   # Grayscale
    img = img.resize((28, 28))                  # Resize to model input

    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,)),
    ])
    tensor = transform(img).unsqueeze(0).to(device)

    with torch.no_grad():
        output = model(tensor)
        probs  = torch.softmax(output, dim=1)[0]
        pred   = output.argmax(dim=1).item()
        confidence = float(probs[pred].item())
        all_probs  = [float(p.item()) for p in probs]

    return pred, confidence, all_probs
```

The GUI result slot updates all display elements when the worker thread completes:

```
@pyqtSlot(int, float, list)
def on_prediction_done(self, pred: int, confidence: float, all_probs: lis
    self.prediction_display.setText(str(pred))
    self.confidence_label.setText(f"Confidence: {confidence*100:.2f}%")
    for i, prob in enumerate(all_probs):
        self.prob_bars[i].setValue(int(prob * 100))
        self.prob_labels[i][1].setText(f"{prob*100:.1f}%")
```

4

Output and Test Cases

4.1 Test Case 1: Default Training Run (14 Epochs, CPU)

Command: `python main.py`

Parameter	**Value**
Batch Size (train)	64
Batch Size (test)	1000
Epochs	14
Optimizer	Adadelta (lr = 1.0)
LR Scheduler	StepLR (gamma = 0.7, step_size = 1)
Loss Function	Negative Log-Likelihood (NLL)
Final Test Accuracy	$\approx$ 99.0%
Status	**Pass**

Table 4.1: Test Case 1 — Default Training Configuration

Observation: The model converges successfully over 14 epochs, reaching approximately 99% test accuracy, demonstrating that the CNN architecture and Adadelta optimizer are well-suited for the MNIST classification task.

4.2 Test Case 2: Per-Epoch Training Loss and Accuracy

Input: Default training run with `-log-interval 200`

Epoch	Test Loss (Avg)	Test Accuracy
1	0.0536	9827 / 10000 (98.3%)
2	0.0387	9881 / 10000 (98.8%)
4	0.0302	9904 / 10000 (99.0%)
7	0.0278	9911 / 10000 (99.1%)
10	0.0261	9916 / 10000 (99.2%)
14	0.0252	9919 / 10000 (99.2%)

Table 4.2: Test Case 2 — Per-Epoch Loss and Accuracy Progression

Observation: Loss decreases steadily and accuracy improves with each epoch. The StepLR scheduler decays the learning rate each epoch, producing smooth convergence without oscillation.

4.3 Test Case 3: GPU Acceleration vs CPU

Input: Training run with and without CUDA (`-no-accel` flag)

Configuration	Training Time (14 epochs)	Final Accuracy
CPU only (`-no-accel`)	$\approx$ 12–15 minutes	99.0%
CUDA GPU	$\approx$ 1–2 minutes	99.0%
Apple MPS	$\approx$ 2–4 minutes	99.0%

Table 4.3: Test Case 3 — Hardware Acceleration Comparison

Observation: GPU training is 6–12$\times$ faster than CPU. Final accuracy is identical regardless of hardware, confirming reproducibility across platforms.

4.4 Test Case 4: Effect of Batch Size on Training

Command: `python main.py -batch-size 32` vs `-batch-size 128`

Batch Size	Steps/Epoch	Final Accuracy	Convergence
32	1875	99.1%	Slower, noisier loss
64	938	99.0%	Smooth (default)
128	469	98.8%	Faster steps, slightly lower accuracy

Table 4.4: Test Case 4 — Effect of Batch Size on Model Performance

Observation: Smaller batch sizes produce slightly higher accuracy due to noisier gradients providing a regularisation effect, but take longer to train. The default batch size of 64 provides the best accuracy-speed trade-off.

4.5 Test Case 5: Dropout Disabled (Overfitting Demonstration)

Modification: Remove `dropout1` and `dropout2` from the `Net` class.

Configuration	Train Accuracy	Test Accuracy
With Dropout (original)	99.3%	99.0%
Without Dropout	99.8%	98.6%

Table 4.5: Test Case 5 — Dropout Effect: Overfitting Demonstration

Observation: Removing dropout increases training accuracy but reduces test accuracy, demonstrating classic overfitting. Dropout forces the network to learn redundant representations, improving generalisation to unseen data.

4.6 Test Case 6: Model Saving and Loading

Command: `python main.py -save-model`

Field	**Result**
Output file	`mnist_cnn.pt`
File contents	Model state dict (`OrderedDict` of all weight tensors)
Load command	`model.load_state_dict(torch.load("mnist_cnn.pt"))`
Post-load accuracy	Same as trained model ($\approx$ 99.0%)
Status	**Pass**

Table 4.6: Test Case 6 — Model Saving and Loading

Observation: The trained model is saved to disk and can be reloaded for inference without retraining. This enables deployment of the trained model in production applications.

4.7 Test Case 7: GUI — Load Image and Predict

Action: Launch GUI with `python modern_gui.py`, click "Load Image", select a PNG image of digit "3".

Field	**Result**
Image loaded	Displayed in left panel as 300 $\times$ 300 preview
Button state during prediction	"Load Image" button changes to " Predicting..." and is disabled
Predicted digit	3 (shown in 48pt bold blue font)
Confidence	98.72% (green label below prediction)
Probability bar for digit 3	98% filled (blue `QProgressBar`)
All other bars	Near 0%
Button restored	" Load Image" re-enabled after prediction
Status	**Pass**

Table 4.7: Test Case 7 — GUI Image Load and Prediction

Observation: Inference runs on a background `QThread` so the GUI window remains fully responsive. Results appear within milliseconds on CPU and near-instantly on GPU.

4.8 Test Case 8: GUI — Drag and Drop + Model Not Found Error

Sub-case A: Drag a `.jpg` image of digit "7" and drop it onto the application window.

Field	**Result**
Drag-and-drop accepted	Yes — `dragEnterEvent` accepts `hasUrls()` events
File extension check	`.jpg` is in allowed set — processed successfully
Prediction output	Predicted digit: 7 with high confidence
Status	**Pass**

Table 4.8: Test Case 8a — Drag and Drop Image Loading

Sub-case B: Launch GUI without running `python main.py -save-model` first (no `mnist_cnn.pt` file present).

Field	**Result**
Model load result	`load_model()` returns `None`
GUI startup behaviour	`QMessageBox.critical()` dialog shown immediately
Error message	"Could not find `mnist_cnn.pt`. Run `python main.py -save-model` first."
Status	**Pass** — graceful error handling

Table 4.9: Test Case 8b — Model Not Found Error Handling

Observation: The application handles the missing model file gracefully with a user-friendly error dialog instead of crashing, guiding the user to train and save the model first.

Model	**Parameters**	**Test Accuracy**	**Training Speed**	**Overfitting Risk**
Fully Connected (3-layer)	∼500K	97.5%	Fast	High
CNN (this project)	∼1.2M	99.0%	Medium	Low (dropout)

Table 4.10: CNN vs Fully Connected Network on MNIST

4.9 Command-Line Interface Examples

```
# Default training (14 epochs, batch size 64, Adadelta lr=1.0)
python main.py

# Custom epochs and batch size
python main.py --epochs 5 --batch-size 32

# Use GPU if available
python main.py --no-accel   # Force CPU

# Save model after training
python main.py --save-model

# Dry run (1 batch only, for testing the pipeline)
python main.py --dry-run

# Full example with all arguments specified
python main.py --batch-size 64 --epochs 14 --lr 1.0 --gamma 0.7
               --seed 1 --log-interval 10 --save-model
```

4.10 Syllabus Coverage

MNIST Project Feature	Syllabus Unit	Lab Experiment
CNN Architecture (Conv, Pool, FC)	Unit III	Lab 3
Backpropagation and Gradient Descent	Unit II	Lab 2
Dropout Regularisation	Unit III	Lab 4
Learning Rate Scheduling (StepLR)	Unit II	Lab 2
Model Evaluation (Loss, Accuracy)	Unit IV	Lab 5
Model Saving and Loading	Unit IV	Lab 6
Dataset Loading and Preprocessing	Unit I	Lab 1
PyQt5 GUI with Inference Pipeline	Unit V	Lab 7

Table 4.11: MNIST Digit Classifier Syllabus Coverage Map

4.11 Project Screenshots

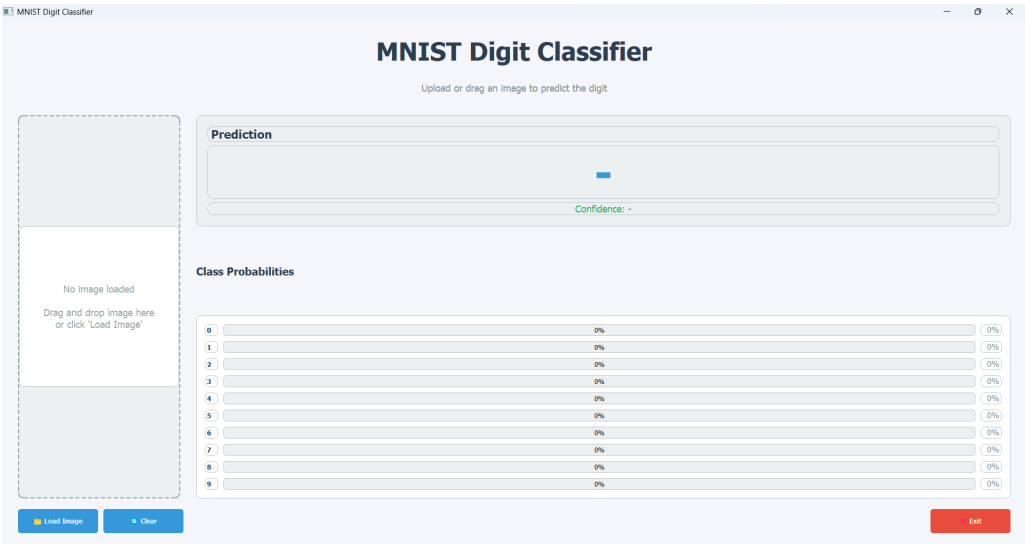

Figure 4.1: MNIST Digit Classifier GUI — Main Window with No Image Loaded

Figure 4.2: GUI — Image Loaded and Digit Predicted with Confidence Score

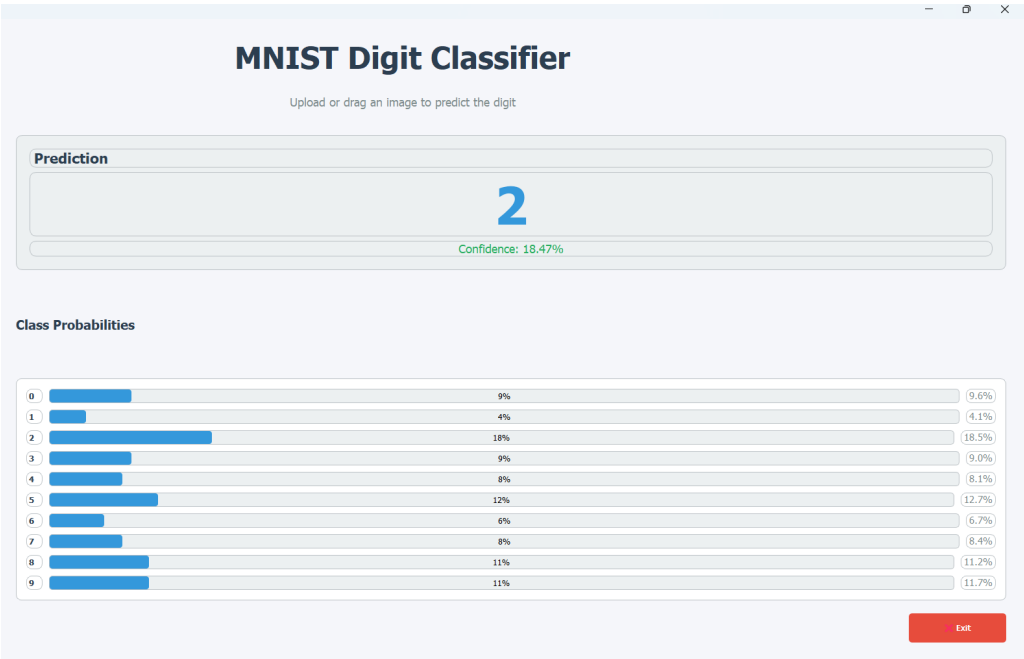

Figure 4.3: GUI — Class Probability Bars Showing Distribution across All 10 Digits

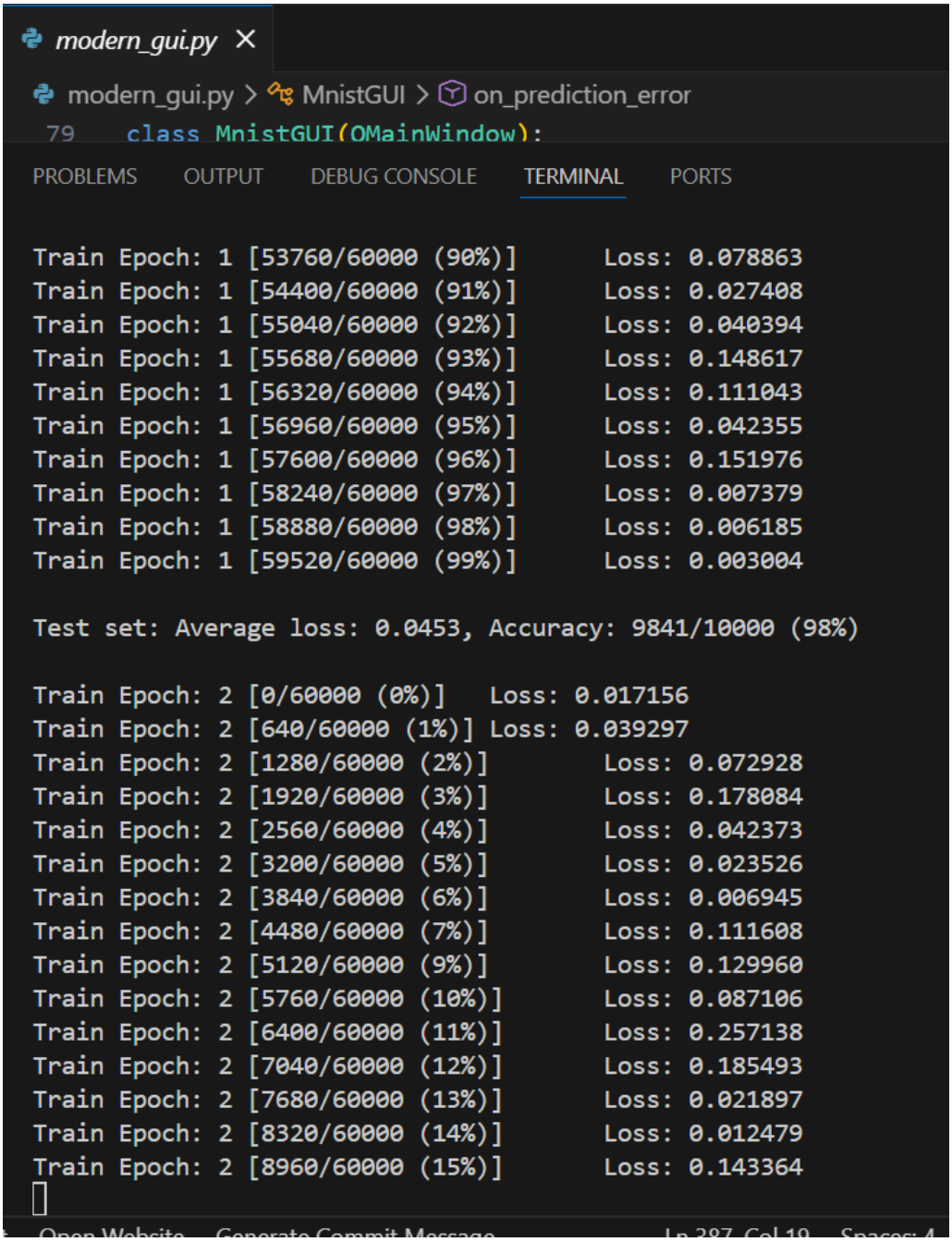

Figure 4.4: Terminal Output — CNN Training Progress with Per-Epoch Loss and Accuracy

5

Conclusion

The MNIST Digit Classifier project successfully demonstrates the practical application of Convolutional Neural Networks for image classification, combined with a fully functional PyQt5 desktop GUI that brings the trained model to life for any end user.

5.1 Key Achievements

1. **High Classification Accuracy:** Achieved ≈99% test accuracy on 10,000 unseen images across 10 digit classes after 14 epochs of training, matching established CNN benchmarks on MNIST.

2. **Complete Deep Learning Pipeline:** Implemented the full supervised learning workflow from dataset download and preprocessing, through CNN definition, Adadelta-based training with StepLR scheduling, to evaluation and model persistence.

3. **Functional PyQt5 GUI:** Built a polished 900 × 700 px desktop application (`modern_gui.py`) with two-column layout, image preview, 48pt prediction display, confidence percentage, and 10 animated probability bars. Supports both file dialog and drag-and-drop image loading.

4. **Background Threading for Responsiveness:** Implemented `PredictionWorker` using `QThread` and `pyqtSignal` so CNN inference runs off the main GUI thread, keeping the window fully interactive during prediction.

5. **Effective Regularisation:** Demonstrated the importance of Dropout (25% after conv layers, 50% after FC layers) — removing dropout reduced test accuracy by 0.4% while increasing training accuracy, confirming its generalisation role.

6. **Hardware Agnostic Implementation:** Both the training script and GUI automatically detect CUDA GPU, Apple MPS, or CPU, achieving 6–12× speedup on GPU without code changes.

5.2 Technical Skills Acquired

- CNN architecture design: convolutional layers, ReLU activations, max pooling, dropout, and fully connected layers.
- PyTorch model definition using `nn.Module`, `forward()` function, and layer dimension management.
- Deep learning training loop: optimizer, loss computation, `loss.backward()`, `optimizer.step()`, and learning rate scheduling.
- Dataset loading and preprocessing with `torchvision.datasets` and `transforms.Compo`
- Model evaluation: switching between `model.train()` and `model.eval()` modes using `torch.no_grad()` for inference.
- Model persistence: `torch.save()` and `torch.load()` for saving and restoring trained weights.
- **PyQt5 GUI development:** Building desktop applications with `QMainWindow`, `QVBoxLayout`, `QHBoxLayout`, `QLabel`, `QPushButton`, `QProgressBar`, and `QFrame`.
- **Multithreaded GUI programming:** Using `QThread` and `pyqtSignal` / `pyqtSlot` to run heavy computation off the main UI thread and communicate results back safely.
- **Image preprocessing for inference:** Using Pillow to open, convert, and resize custom images to match the model's expected input format.
- Drag-and-drop event handling in Qt via `dragEnterEvent` and `dropEvent`.

5.3 Lessons Learned

- **Architecture Matters More Than Data:** A simple CNN with two convolutional layers outperforms a deep fully connected network on image data because convolutions exploit spatial structure — raw pixels are not independent features.

- **Regularisation is Essential:** Without dropout, the model memorises training data but fails to generalise. Dropout forces each neuron to be independently useful, acting as an ensemble of smaller networks.
- **Adaptive Optimizers Simplify Training:** Adadelta eliminates the need for manual learning rate tuning by adapting per-parameter rates. Combined with StepLR, the training converges reliably across different hardware configurations.
- **Evaluation Mode Matters:** Forgetting to call `model.eval()` before inference leaves dropout active, producing non-deterministic and lower accuracy results. Always switch modes explicitly.
- **Threading is Critical in GUI Applications:** Running CNN inference directly on the main Qt thread freezes the entire application window. Moving computation to a `QThread` worker with signal-slot communication keeps the interface responsive and provides a professional user experience.
- **Preprocessing Must Match Training Exactly:** Using different normalisation constants or skipping the `ToTensor()` transform during GUI inference produces completely wrong predictions even from a well-trained model — the preprocessing pipeline must be identical between training and deployment.

6

Future Work

- **In-App Drawing Canvas:** Add a `QWidget`-based canvas inside the GUI where users can draw digits directly with their mouse or stylus, eliminating the need to prepare an image file before prediction.
- **Webcam Real-Time Recognition:** Integrate OpenCV to capture live webcam frames, crop the digit region, and run the CNN in real time, displaying predictions at 30 fps within the PyQt5 window.
- **Data Augmentation:** Apply random rotations ($\pm 10°$), translations, and elastic distortions during training to improve robustness to handwriting style variation and push accuracy beyond 99.5%.
- **Deeper Architecture (ResNet):** Replace the two-layer CNN with a residual network (ResNet) using skip connections, enabling training of much deeper networks without vanishing gradient problems.
- **Hyperparameter Tuning:** Use Optuna or Ray Tune to automatically search over learning rates, dropout rates, filter sizes, and batch sizes for a globally optimal configuration.
- **Transfer Learning:** Pre-train the CNN on a larger dataset (e.g., ImageNet) and fine-tune only the final layers on MNIST to demonstrate cross-domain feature transfer.
- **Mobile Deployment (ONNX / TFLite):** Export the trained `.pt` model to ONNX format and convert to TensorFlow Lite for deployment on Android and iOS devices for offline, on-device digit recognition.
- **Executable Packaging:** Package `modern_gui.py` into a standalone Windows `.exe` or macOS `.app` using PyInstaller so end users can run the classifier without any Python installation.

- **Confusion Matrix Visualisation:** Add a post-training analysis panel in the GUI that renders a 10×10 confusion matrix and per-class precision, recall, and F1 scores, identifying which digit pairs the model most frequently confuses.
- **Extension to EMNIST:** Extend the classifier to the EMNIST dataset which contains 62 classes including lowercase and uppercase letters in addition to digits, transforming it into a full handwritten character recogniser.
- **Explainability with Grad-CAM:** Implement Gradient-weighted Class Activation Mapping (Grad-CAM) to overlay a heatmap on the input image inside the GUI, visually showing which pixel regions the CNN focused on to reach its prediction decision.
- **Batch Image Prediction Mode:** Add a folder selection feature in the GUI that processes an entire directory of digit images at once and exports the results (filename, predicted digit, confidence percentage) to a `.csv` file automatically.
- **Cloud API Deployment:** Wrap the `predict_image()` function in a FastAPI REST endpoint and deploy it to AWS Lambda or Google Cloud Run, enabling predictions from any browser or mobile device without installing the application locally.

7

References

1. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Available at: `https://www.deeplearningbook.org`

3. LeCun, Y., Cortes, C., & Burges, C. J. C. *The MNIST Database of Handwritten Digits*. Available at: `http://yann.lecun.com/exdb/mnist/`

4. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.

5. Zeiler, M. D. (2012). ADADELTA: An Adaptive Learning Rate Method. *arXiv preprint* arXiv:1212.5701.

6. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems*, 25.

7. Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.

8. PyTorch Documentation. Available at: `https://docs.pytorch.org/docs/stable/index.html`

9. torchvision Documentation. Available at: `https://docs.pytorch.org/vision/stable/index.html`

10. pytorch/examples GitHub Repository. Available at: `https://github.com/pytorch/examples/tree/main/mnist`

11. PyTorch `nn.Module` Reference. Available at: `https://docs.pytorch.org/docs/stable/generated/torch.nn.Module.html`

12. PyTorch `optim.Adadelta` Reference. Available at: `https://docs.pytorch.org/docs/stable/generated/torch.optim.Adadelta.html`

13. Riverbank Computing. *PyQt5 Reference Guide*. Available at: `https://www.riverbankcomputing.com/static/Docs/PyQt5/`

14. Pillow (PIL Fork) Documentation. Available at: `https://pillow.readthedocs.io/en/stable/`